Reproducibility Engineering
Summer Term 2026
Chair of Scalable Database Systems
Prof. Dr.-Ing. Stefanie Scherzinger, Stefan Klessinger, Christoph Köhnen

UNIVERSITY OF PASSAU
Faculty of Computer Science and Mathematics

Exercise Sheet 4

Lab Sessions: May 21/22, 2026

In this exercise sheet, we focus on notions of structural equivalence, on reporting and on documentation. In particular, we aim to make our reproduction package **gold standard**, to the point where even the charts and the final paper are created dynamically.

1 Preparation

For this exercise, we provide multiple files in the RepEng FIMGit Repository (directory `LabSession4/`). Update your local repository that you cloned in the first lab session:

1. Open a terminal in your local repository.
2. Pull the latest changes using `git pull`.

2 Structural Equivalence

When checking reproducibility, we often need to check whether two output files are equivalent. However, what we consider equivalent is highly context-dependent. When working with XML, for example, the language XPath provides multiple notions of equality.

2.1 Preparation: Docker Environment

In the RepEng repository, the subdirectory `LabSession4/2_structural_equivalence/` contains a `Dockerfile` and a sample XML file. The Dockerfile specifies an Ubuntu image that installs `Xidel`, a command line tool that evaluates XPath queries on input data.

Build the image from the Dockerfile and start a temporary interactive container:

Terminal Session bash

```
user@host$ cd LabSession4/2_structural_equivalence/
user@host$ docker build -t xidel-env .
user@host$ docker run --rm -it xidel-env
```

2.2 Experiment Data

Consider the following XML data (`experiments.xml`) provided in the repository:

XML Document XML

```xml
<experiments>
  <exp id="A"><time unit="s">42</time></exp>
  <exp id="B"><time>42</time></exp>
  <exp id="C"><time>42</time></exp>
  <exp id="D">
    <time>40</time>
    <time>42</time>
  </exp>
  <exp id="E"><time> 42 </time></exp>
</experiments>
```

This XML snippet contains measurements from five experiments, each uniquely identified by an `id` attribute. The data has structural variations: Experiment A is the only entry that specifies a `unit` attribute. Experiments A, B, C and E record a single `<time>` element, whereas experiment D records a sequence of `<time>` values. Note that in experiment E, the value of `time` contains whitespaces.

2.3 Tooling: Xidel

Inside the container, we use `Xidel`[1] to evaluate XPath expressions on `experiments.xml`:

Terminal Session bash

```
user@container$ xidel experiments.xml -se "<XPath expression>"
```

Mind the quotation marks around the XPath expression.

The parameters take the following effects:

- `-s`: Suppresses metadata and status messages. Only the query result is printed.
- `-e`: Evaluates the XPath expression on the provided input data.

2.4 Comparing Equality Notions

We compare the following notions of equality in XPath[2]. Evaluate each of the provided sample XPath expressions on the data and explain the obtained results.

1. **Value Comparison (`eq`):** Compares the exact text value of two single nodes.

   ```
   //exp[@id='A']/time eq //exp[@id='E']/time
   ```

[1] https://www.videlibri.de/xidel.html

[2] For a precise and detailed definition of each operator, consult https://www.w3.org/TR/xpath-30/#id-comparisons and https://www.w3.org/TR/xpath-functions-30/#func-deep-equal.

2. **Structural Comparison (`deep-equal()`):** Checks whether two nodes have the exact same structure, including all attributes, text content, and child nodes.

```
deep-equal(//exp[@id='A']/time, //exp[@id='B']/time)
```

```
deep-equal(//exp[@id='A']/time, //exp[@id='E']/time)
```

3. **Node Comparison (`is`):** Checks whether two expressions point to the exact same physical element in the document tree. Even if two nodes look perfectly identical, they are not considered the same node if they reside at different paths.

```
//exp[@id='B']/time is //exp[@id='C']/time
```

4. **General Comparison (`=`):** A comparison that can handle sequences. It evaluates to `true` if there are any equal values between two sequences of nodes.

```
//exp[@id='B']/time = //exp[@id='D']/time
```

Perform further comparisons on the sample data using the `eq`, `deep-equal()`, `is`, and `=` operators. Find and discuss suitable practical scenarios for each notion of equality.

3 LaTeX in Docker: Freezing Dependencies

A central aspect of making your experiments reproducible is describing your experimental setup and the obtained results. For writing such a paper, LaTeX is the predominant tool in computer science. It has a modular design where the core system provides only a minimal foundation, that can be extended with external packages.

To compile our paper, we need LaTeX inside our Docker container. *TeX Live* is a popular LaTeX distribution that offers different bundles, such as the *full* bundle, which contains virtually all available packages, or the *base* bundle which only contains the most frequently used packages. Usually, only a small fraction of the LaTeX packages installed by these distributions is actually needed, leading to unnecessarily large Docker image sizes. Thus, installing only necessary packages through a lightweight distribution such as *TinyTeX* helps to keep image sizes small.

We will next compare a Docker image using a *base* TeX Live installation against an image with a minimal TinyTeX setup.

3.1 TeX Live Installation

Create a Dockerfile based on Ubuntu 24.04 that installs the following TeX Live packages via `apt`: `texlive-latex-base`, `texlive-latex-extra`, `texlive-fonts-recommended`.

Build the image and ensure that it successfully compiles the LaTeX skeleton provided at `LabSession4/4_reporting/experiment.tex` using `pdflatex`:

```
pdflatex experiment.tex
pdflatex experiment.tex
```

Note: To resolve references such as `\ref{fig:results_plot}`, `pdflatex` needs to be run twice.

3.2 TinyTeX Installation

Create a second Dockerfile that, instead of the TeX Live packages, installs the lightweight TinyTeX distribution.

Setting up TinyTeX requires the packages `ca-certificates`, `perl`, `wget`, and `xz-utils`. After installing them, set up TinyTex in the Dockerfile as follows:

Dockerfile

```
[...]
# Download and install TinyTeX
RUN wget -qO- "https://tinytex.yihui.org/install-bin-unix.sh" | sh

# Add TinyTeX to PATH
ENV PATH="/root/.TinyTeX/bin/x86_64-linux:${PATH}"
[...]
```

TinyTeX can download packages on the fly, but this makes builds non-deterministic. Instead, extend your Dockerfile to *freeze* the package set: Explicitly specify every package required for your paper using `tlmgr install` in the Dockerfile.

Make sure to install at least the following packages: `amsmath`, `graphicx`, `hyperref` (plus any other packages your specific document requires):

Dockerfile

```
[...]
RUN tlmgr install amsmath graphicx hyperref
[...]
```

Afterwards, build the image and ensure that it successfully compiles the LaTeX skeleton provided at `LabSession4/4_reporting/experiment.tex` using `pdflatex`:

```
pdflatex experiment.tex
pdflatex experiment.tex
```

3.3 Size Comparison

After building both the TeX Live and TinyTeX images and confirming that they successfully compile your LaTeX document, compare the image sizes by filling in the following table.

Docker Image	**ubuntu:24.04**	**TeX Live Base**	**TinyTeX**
Image Size (MB)			

Hints:

- To download the Ubuntu base image, run `docker pull ubuntu:24.04`
- To obtain information on Docker image sizes, run `docker images`

4 Automated Reporting

4.1 Documenting Experiments

In this exercise, you will create a small LaTeX document that presents your `pplease` experiment. Fill in the provided LaTeX skeleton from the repository (`LabSession4/4_reporting/experiment.tex`).

Make sure your document covers:

- **Hypothesis/Research Question:** Formulate your own hypothesis or research question for your `pplease` experiment.
- **Experiment Setup:**
  - Describe the implementation of `pplease` (e.g., Python version, usage of LLMs).
  - Describe the execution environment (Docker container and host system).
  - Describe the input data (you may choose your own).
  - Describe what is measured, how often, and how (e.g., whether you measure runtime, you may want to perform several runs and take the mean or median).
- **Results:**
  - Include a chart generated dynamically from the data through a Python script. The chart must match your hypothesis or research question.
  - In the text, describe the results, referring to the chart.
- **Discussion:** Discuss the results with respect to the hypothesis/research question.

After creating your document, ask a fellow student to provide feedback. Revise your document based on your peer's feedback, then ask your tutor for feedback.

4.2 Reproduction Package Integration

Now, integrate the LaTeX compilation step into the reproduction package you created in the previous exercise sheet by following these steps:

1. **Extend your Dockerfile:** Merge the *TinyTeX* installation steps from Exercise 3 into your `Dockerfile` from Sheet 3. Your container must now be capable of running the Python experiments *and* compiling LaTeX documents.

2. **Link Data to LaTeX:** Update your `experiment.tex` file to load the chart from a fixed path (e.g., using `\includegraphics{results/chart.pdf}`).

3. **Automate the Pipeline:** Update your `run_experiment.sh` dispatcher script. The script must now sequentially:
   - Execute the `pplease` experiments.
   - Generate the chart/data and ensure it is saved or moved to the exact path expected by your LaTeX document.
   - Compile the paper using `pdflatex`:

     ```
     pdflatex -interaction=nonstopmode -halt-on-error experiment.tex
     ```

     Note: The parameters `-interaction=nonstopmode` and `-halt-on-error` are not strictly necessary but a good practice in automated pipelines, since they enable swift termination in case of errors.

4.2.1 Evaluating Repeatability

Run your reproduction package to execute the experiment and generate the final PDF. Inspect the document to ensure everything, including the figure, was generated correctly.

Afterwards, run the entire reproduction package a second time without changing any code or data, and generate the PDF again. Afterwards, compute the MD5 hash of both PDF files using the command:

```
md5sum <your_paper>.pdf
```

Are the MD5 hashes identical? If not, why might the PDF files differ bitwise even if the underlying experimental data is exactly the same? Discuss whether the result is repeatable in this case.

5 XPath and Docker Images (Antwort-Wahl-Verfahren)

In this exercise, each question has exactly one correct answer. Mark the correct answer.

(a) Consider the following XML snippet:

XML Document XML

```xml
<experiments>
  <exp id="1"><time>42</time></exp>
  <exp id="2">
    <time>41</time>
    <time>42</time>
  </exp>
</experiments>
```

Which of the following XPath expressions evaluates to **true**?

- ○ `//exp[@id='1']/time eq //exp[@id='2']/time`
- ○ `//exp[@id='1']/time = //exp[@id='2']/time`
- ○ `deep-equal(//exp[@id='1']/time, //exp[@id='2']/time)`
- ○ `//exp[@id='1'] is //exp[@id='2']`
- ○ None of these options

(b) Consider the following XML snippet:

XML Document XML

```xml
<experiments>
  <exp id="1"><time>42</time></exp>
  <exp id="2"><time>42</time></exp>
</experiments>
```

Which of the following XPath expressions evaluates to **false**?

- ○ `//exp[@id='1']/time eq //exp[@id='2']/time`
- ○ `//exp[@id='1']/time = //exp[@id='2']/time`
- ○ `deep-equal(//exp[@id='1']/time, //exp[@id='2']/time)`
- ○ `//exp[@id='1'] is //exp[@id='2']`
- ○ None of these options

(c) Consider the following Dockerfile snippets.

A)

```
FROM ubuntu:24.04
RUN apt-get update && \
    apt-get install -y \
    --no-install-recommends \
    python3=3.12.3-0ubuntu2.1 \
    && rm -rf /var/lib/apt/lists/*
```

B)

```
FROM ubuntu:24.04
RUN apt-get update && \
    apt-get install -y \
    --no-install-recommends \
    python3 \
    && rm -rf /var/lib/apt/lists/*
```

C)

```
FROM ubuntu:latest
RUN apt-get update && \
    apt-get install -y \
    --no-install-recommends \
    python3=3.12.3-0ubuntu2.1 \
    && rm -rf /var/lib/apt/lists/*
```

D)

```
FROM ubuntu:24.04
RUN apt-get update && \
    apt-get install -y \
    --no-install-recommends \
    python3=3.12.3-0ubuntu2.1 \
    python3-pip=24.0+dfsg-1ubuntu1.3 \
    && rm -rf /var/lib/apt/lists/*
RUN pip install pandas
```

How many of the Dockerfile snippets above ensure that the resulting image will remain stable over a long time w.r.t. pulled dependencies?

○ 0 ○ 1 ○ 2 ○ 3 ○ 4